Python Dictionary Internals

Akshay M
Software Engineer, Google

Overview

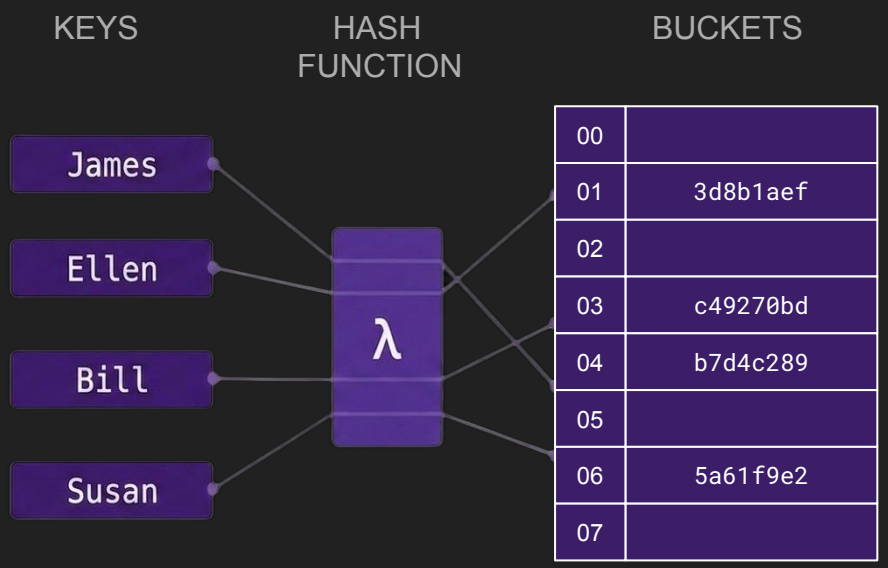
- The Dictionary
- Dictionaries as Hash Tables
- How Hashing Works
- Collisions
- Collision Handling
- Dictionary in Python
- Hashing in Python
- Python Probing Strategy
- Memory Layout
- Performance trade-offs
- Real World Implications

The Dictionary

- Key → Value mapping
- Insert, lookup, delete, and update operations
- Abstract data structure

Key	Value
James	25
Ellen	29
Bill	27

Dictionaries as Hash Tables



How Hashing Works

key = “james”

hash(key) = 37

bucket_size = 5

index = hash(key) % buckets

= 37 % 5

= 2

00	
01	
02	james
03	
04	

Integers

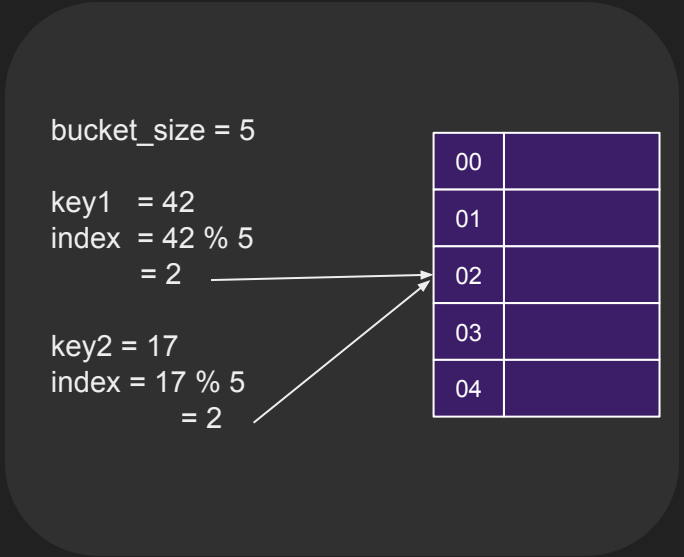
- hash(x) = x

Strings

- Polynomial Rolling Hash:
  hash(“abc”) = 'a' (97) x 31^2 + 'b' (98) x 31^1 + 'c' (99) x 31^0

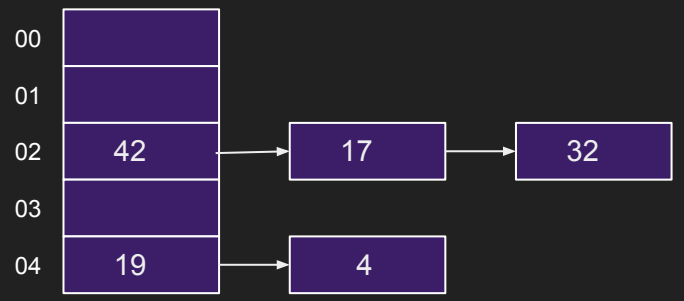
  = 96,354

Collisions

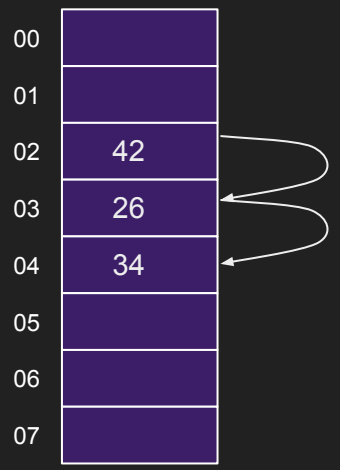

bucket_size = 5
key1 = 42
index = 42 % 5
= 2
key2 = 17
index = 17 % 5
= 2
00
01
02
03
04

Collision Handling

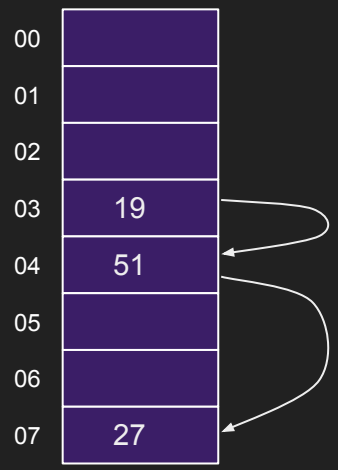
Chaining



Open Addressing



Linear Probing



Quadratic Probing

$$j_i = (j_0 + i^2) \pmod{N}$$

Dictionary in Python

- Hash table–based implementation
- Open addressing with efficient probing
- Key → hash → index → probe → entry
- Optimized for fast, memory-efficient lookups
- Preserves insertion order
- Load factor of ⅔

Hashing in Python

Integers

hash(x) = x % M

M = Mersenne prime: $2^{61} - 1$

Strings

Uses SipHash function: prevents hash-flooding DoS attacks

SipHash Visualiser: siphash.akshayxml.com

Float

Converts float f to m x 2^e,
And eventually to integer format.

Tuple

hash((a, b)) = mix(hash(a), hash(b))

Custom Objects

```
class A:
    def __hash__(self):
        return 123
```

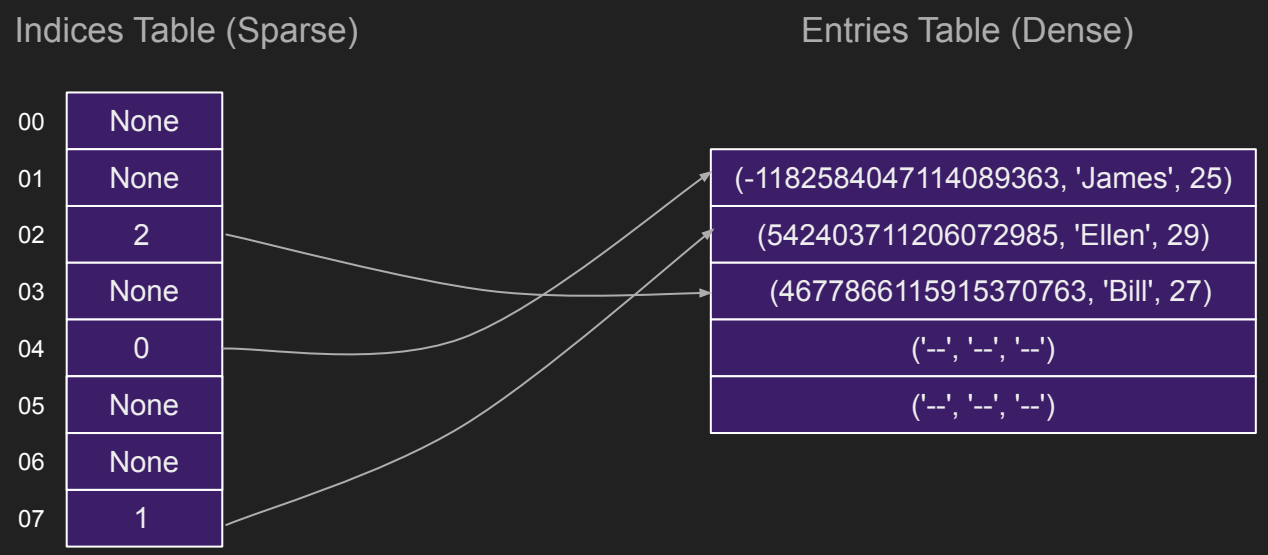
Python Probing Strategy

$$j_{next} = (5 \times j_{current} + 1 + \text{perturb}) \pmod{N}$$
$$perturb >>= 5$$

N is the table size
perturb initially equals the full hash value

- The multiplier (x5): Prevents primary clustering
- The addition (+1): Ensures sequence doesn't get stuck at 0
- The variable (+perturb): Prevents secondary clustering

Memory Layout



Performance trade-offs

Space vs. Speed
(The Load Factor)

Open Addressing
vs.
Separate Chaining

Security
vs
Hashing Speed
(Hash DoS Protection)

Speed vs. Hash Distribution
(Power of 2 table size)

Real World Implications

The "Fast" Side

- Cache Locality
- Low load factor
- Fast Collision Resolution
- Efficient indexing (bitwise &)
- Hash stored in entry

The "Slow" Side

- The ⅔ resizing cliff
- Terrible custom __hash__ Implementations
- Frequent deletions
- Large dictionary (cache misses)

Resources

- Raymond Hettinger - Modern Python Dictionaries - A confluence of a dozen great ideas - PyCon 2017
- cpython/Objects/dictobject.c at main
- Python behind the scenes #10: how Python dictionaries work